

How We Invented the LSTM at the Kitchen Table

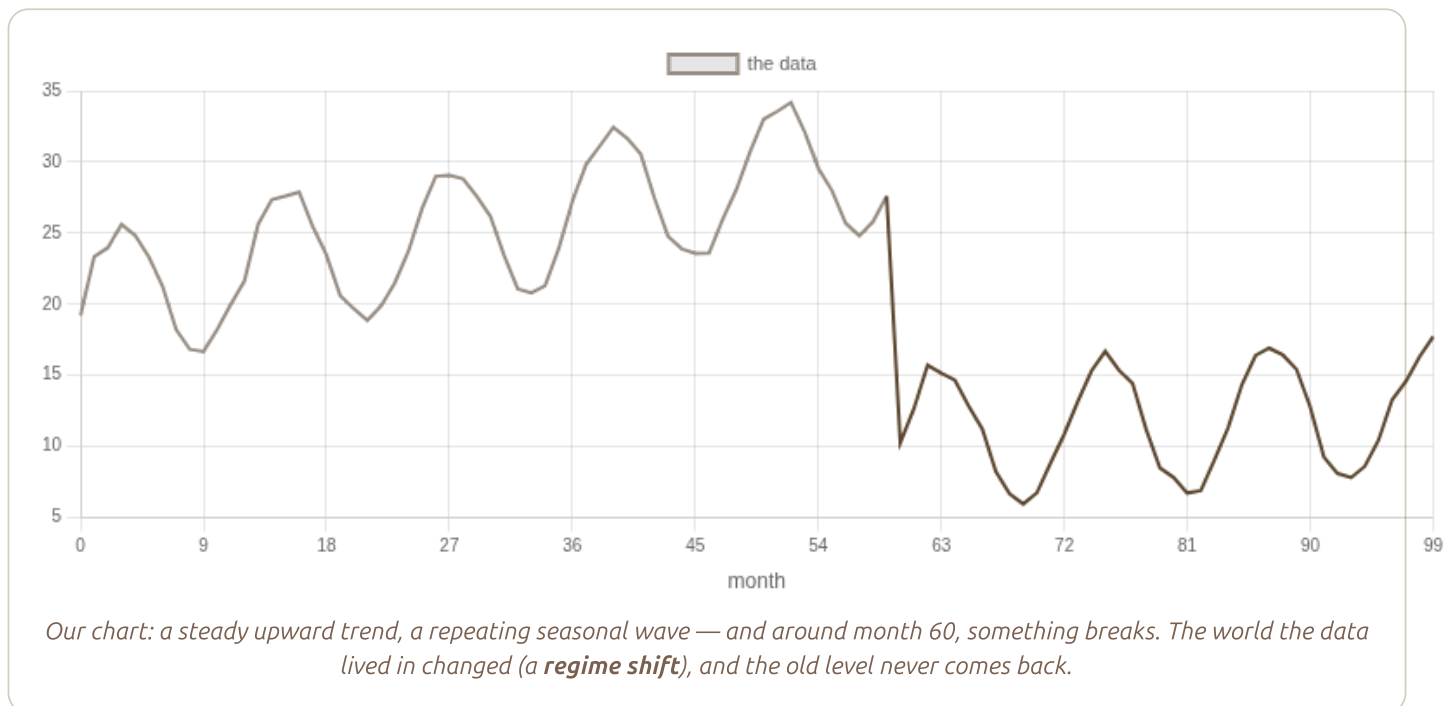
A step-by-step collaborative discovery story: one chart, a whiteboard, a few naive ideas that fail in instructive ways — and the architecture that falls out of fixing them.

Weight and bias names throughout this document follow the exact convention of the companion backpropagation document (e.g. $w_{xo_{t-1}}$, net_o , b_f), so you can move between the two without translating notation. If you are coming from the gate diagrams: the input gate i_t here is the “% Potential Memory” block, and the candidate g_t is the “Potential Long Memory” block.

1 • The Problem & the First Questions

Picture the scene. We are sitting around a wooden kitchen table. In the middle is a large printed chart of a value measured every month for years — it could be a stock price, a temperature, a sales figure. There are markers, coffee, and a whiteboard leaning against the wall. Our job, given everything on this chart, is to **predict the next point**.

Before any equations, we just *look* at it:



Staring at it, the first natural questions come fast:

- “Should tomorrow’s guess mostly depend on **yesterday**? Or on the **whole history**?”

- “There’s clearly a repeating wave — how do we **remember the pattern** from a full cycle ago?”
- “Look at month 60. Everything before it suddenly stopped being relevant. How would a predictor **let go** of the old world?”
- “And whatever we build — can it **learn** these judgements from the data, instead of us hand-tuning them?”

Nobody says “LSTM.” Nobody knows that word yet. We just start with the simplest idea anyone can propose.

2 • Attempt #1 — Only Long-Term Memory

Someone says: “What if we just average everything we’ve ever seen? The average is stable, it uses all the data, and it can’t be fooled by one weird day.”

START SIMPLE — THE PLAIN-ENGLISH IDEA

Our prediction for the next point is just: *add up every value seen so far and divide by how many there are.*

NOW AS A FORMULA — THE CUMULATIVE MEAN

If the values so far are $x_1, x_2, \dots, x_t$, our memory m_t is:

$$m_t = \frac{1}{t} \sum_{k=1}^t x_k$$

The $\sum$ just means “add them all up,” and dividing by t makes it an average.

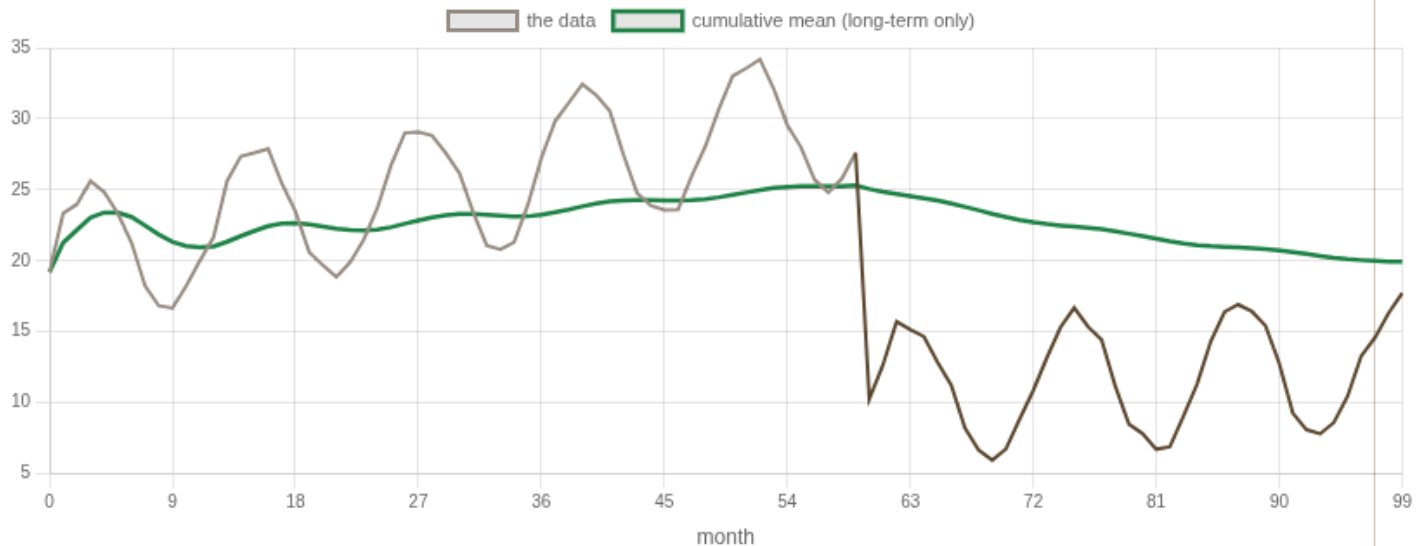
ONE STEP FURTHER — WRITE IT AS AN UPDATE RULE

We don’t actually need to re-add the whole history each day. A little algebra turns the same average into a *running update*:

$$m_t = m_{t-1} + \frac{1}{t}(x_t - m_{t-1})$$

Read it aloud: “*new memory = old memory, nudged toward today’s value by a shrinking amount $\frac{1}{t}$.*” This form matters, because it is our first **recurrence** — a memory carried forward and updated. Hold onto that shape; we will see it again.

We try it on the chart:



The cumulative mean (green) versus reality (gray). Watch what happens after the regime shift at month 60.

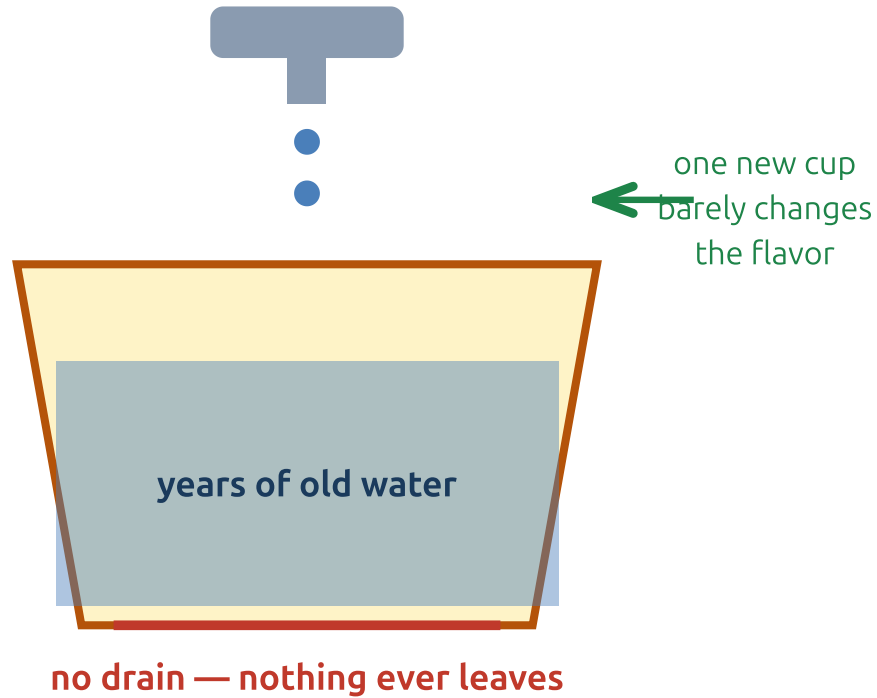
Where it fails — and exactly why. After month 60 the world drops to a new level, but our green line barely reacts. The math says why: at $t = 61$, today's value can only move the memory by $\frac{1}{61}$ of the surprise. Suppose the old average was about 35 and the new reality is about 15:

$$m_{61} = 35 + \frac{1}{61} (15 - 35) \approx 35 - 0.33 = 34.67$$

One month of the new world moved us a third of a point. To actually reach 15 we would need the new regime to outweigh *the entire old history* — years of lag. And it only gets worse with time, because $\frac{1}{t}$ keeps shrinking: the more we have seen, the more stubborn the memory becomes.

The memory bucket (no drain)

every new value pours in



The analogy that sticks: the cumulative mean is a soup pot that every cook in history has poured a cup into — and the pot has *no drain*. By year five, pouring in one cup of a brand-new recipe changes the flavor by almost nothing. The soup doesn't taste like today; it tastes like *the average of everything that ever happened*.

The memory isn't wrong because it remembers — it's wrong because it *cannot forget*.

Whiteboard insight #1

"Remembering everything forever makes us blind to change. We need a way to **selectively forget** — a drain we control."

3 • Attempt #2 — Only Short-Term Memory

Someone across the table pushes back: "*Fine — then let's do the opposite. Forget history. Tomorrow will look like today. Just predict the most recent value.*"

START SIMPLE — THE PLAIN-ENGLISH IDEA

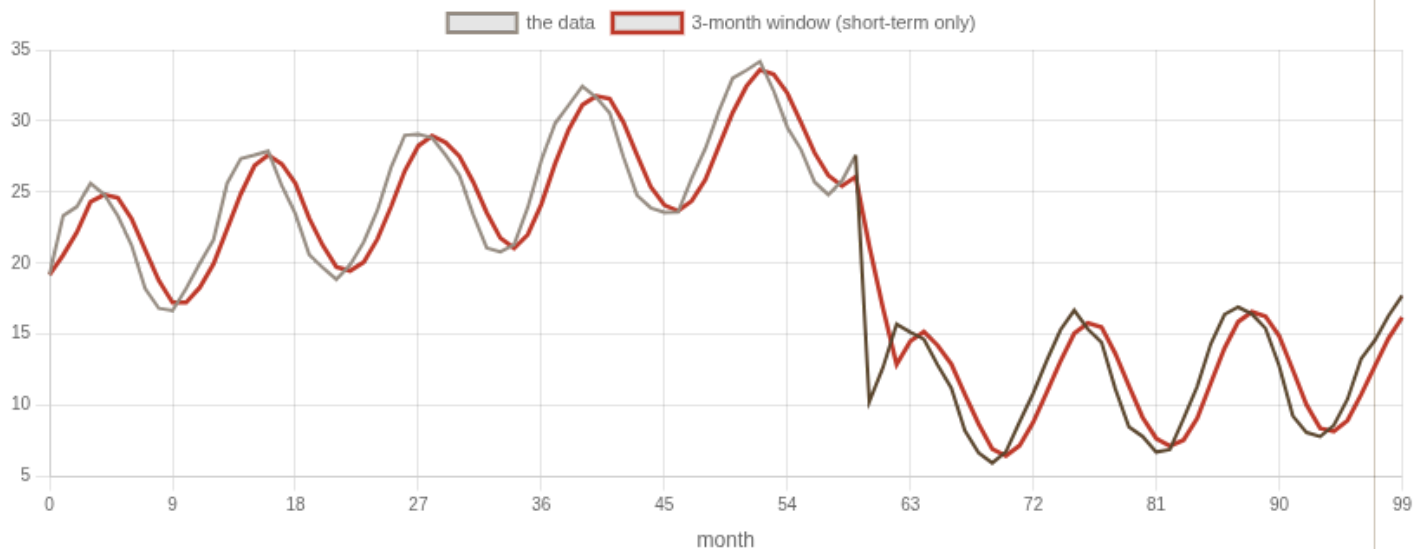
Prediction = the last thing we saw. $\hat{x}_{t+1} = x_t$. Instant reaction, zero lag.

A GENTLER VERSION — A SMALL SLIDING WINDOW

The last value alone is jumpy, so we soften it: average just the last few points — say a window of 3:

$$m_t = \frac{x_t + x_{t-1} + x_{t-2}}{3}$$

Anything older than three steps ago is simply gone. The window slides forward, and old values fall off the back edge.



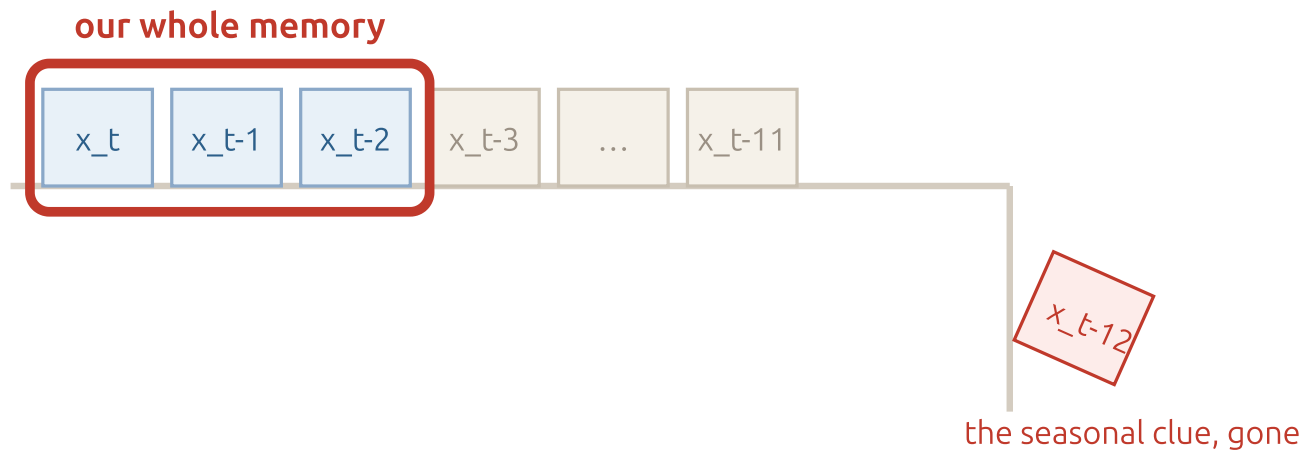
The 3-step window (red) hugs the data and adapts to the regime shift almost instantly. It looks great... until we ask it the questions that need memory.

Where it fails — and exactly why. Look at the seasonal wave in the chart: it repeats every 12 months. To predict a winter dip, we need information from a *full cycle ago*. But our memory is a window of width 3:

$$m_t \text{ depends only on } x_t, x_{t-1}, x_{t-2} \implies \frac{\partial m_t}{\partial x_{t-12}} = 0$$

Read that last expression gently: it is a partial derivative asking “if the value 12 months ago had been different, would our memory change?” — and the answer is exactly zero. Not “a little.” Zero. Whatever happened one season ago has **no path** into our prediction. The window didn’t forget it gradually; it fell off the edge completely.

The window that falls off the edge



The analogy that sticks: short-term-only memory is working from sticky notes on a tiny desk. Every new note pushes the oldest one into the trash. It doesn't matter how important the old note was — “the dip always comes in December” gets thrown out with the same indifference as “Tuesday was cloudy.” Importance plays no role; only *age* does.

This memory isn't wrong because it forgets — it's wrong because it *cannot choose what to keep*.

Whiteboard insight #2

“Reacting instantly makes us amnesiac. We need a way to **preserve important information across many steps** — a shelf where chosen things can sit safely, out of the window's reach.”

4 · The Realization — We Need Both, But With Control

The table goes quiet for a minute. Two attempts, two opposite failures:

Long-term only

Stable, sees the whole history — but **can't forget**, so it lags forever after a change.

Short-term only

Adapts instantly — but **can't choose what to keep**, so anything old is lost regardless of importance.

Then someone says the sentence that changes everything: “*These aren’t competing ideas. They’re two different jobs. What if we keep **two memories** — a protected long-term ledger AND a fast working note — and add **valves** that control what flows between them?*”

New questions immediately pour out, and each will become a component:

- “How much of the ledger should survive each day?” — that valve will become the **forget gate**.
- “What should today write into the ledger, and how much do we trust it?” — the **candidate** and the **input**

1 The Chart 2 Long-Term Only 3 Short-Term Only 4 The Realization 5 Building the Cell 6 Full Diagram 7 The Numbers
8 The History 9 Takeaways

- “And who sets the valves?” — nobody. They must be **learned**: each valve is a tiny neuron whose weights are trained by the same gradient descent that trains everything else.

One design decision before we build: every valve will output a number between 0 and 1 (a “percentage open”), computed from the same two things everywhere: *today’s input* x_t and *yesterday’s working note* h_{t-1} . Same recipe, different learned weights per valve. That uniformity is what makes the design feel inevitable once you see it.

5 • Building the Cell — One Component at a Time

5.0 • The cell state: a conveyor belt for memory

First, the protected ledger itself. We give it a name, c_t (the **cell state**), and one sacred rule: it travels forward on its own line — picture a conveyor belt running across the top of the machine — and each step may only do two gentle things to it: *scale it* and *add to it*. No squashing functions on the belt itself, no weight matrices mangling it in transit.

THE SIMPLEST POSSIBLE FORM FIRST

$$c_t = c_{t-1}$$

“Whatever was on the belt stays on the belt.” A perfect, boring memory. Everything we build next is a *controlled modification* of this line.

A preview of why this belt matters so much: the people who first solved this problem discovered that when memory is repeatedly squashed and re-weighted at every step, the training signal (the gradient) shrinks or explodes exponentially as it flows back through time. The untouched belt is the cure — the full story is in [Section 8](#).

5.1 • The Forget Gate — the drain we control

The intuition, simply: we want to multiply the old ledger by a number between 0 and 1. Multiply by 1 → keep everything. Multiply by 0 → wipe it clean. Multiply by 0.6 → keep 60%. That single multiply is the drain our soup pot never had.

STEP 1 — THE IDEA AS MATH

$$c_t = f_t \times c_{t-1}, \quad 0 \leq f_t \leq 1$$

One number f_t , one multiplication. But who picks f_t ?

STEP 2 — LET THE DATA PICK IT: A WEIGHTED SUM

We compute a raw score from what we can see — today's input and yesterday's working note — each with its own learned weight, plus a bias:

$$\text{net}_f = w_{xf} x_t + w_{hf} h_{t-1} + b_f$$

Symbol by symbol: w_{xf} is "how much today's input should influence forgetting," w_{hf} is "how much yesterday's note should," and b_f is the gate's default lean when both are quiet.

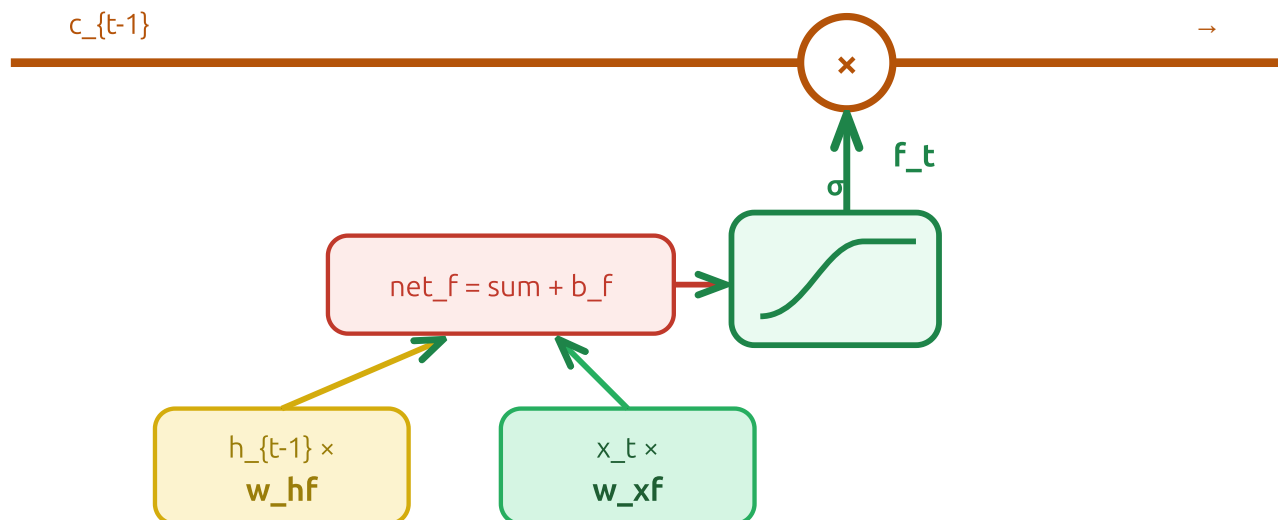
STEP 3 — SQUASH THE SCORE INTO A VALVE POSITION

The raw score can be any number; a valve needs 0-to-1. The sigmoid does exactly that:

$$f_t = \sigma(\text{net}_f) = \frac{1}{1 + e^{-\text{net}_f}}$$

► Rusty on the sigmoid? A 60-second refresher

FORGET GATE



Analogy: f_t is a hand on the drain valve of the soup pot — but the hand belongs to a taster who samples *today's ingredients* (x_t) and *yesterday's tasting notes* (h_{t-1}) before deciding how much old soup to let out. On an ordinary day the valve stays nearly shut ($f_t \approx 1$, keep the soup). The day the menu changes, the taster opens the drain ($f_t \approx 0$) and the pot starts fresh.

Which failure this solves: Attempt #1's. The cumulative mean lagged for years after the regime shift because its memory had no drain. With f_t , one step of "valve open" can erase *any* amount of stale memory — the multiply scales the whole ledger at once, something no amount of averaging-in new data could do.

5.2 · The Input Gate + Candidate — write worthwhile things, in the right amount

The intuition, simply: draining is only half the job — we also need to *write*. And writing is really two separate questions: *what* should today's entry say, and *how much of it* do we trust enough to commit? Two questions, two components.

STEP 1 — THE CANDIDATE: WHAT THE ENTRY SAYS

A proposal for the new content, built the same way as every valve — weighted sum of x_t and h_{t-1} — but squashed with **tanh** instead of sigmoid:

$$\text{net}_g = w_{xg} x_t + w_{hg} h_{t-1} + b_g \quad \Rightarrow \quad g_t = \tanh(\text{net}_g)$$

► Why tanh here and not sigmoid?

STEP 2 — THE INPUT GATE: HOW MUCH OF IT WE TRUST

A valve, identical recipe to the forget gate, with its own weights:

$$\text{net}_i = w_{xi} x_t + w_{hi} h_{t-1} + b_i \quad \Rightarrow \quad i_t = \sigma(\text{net}_i)$$

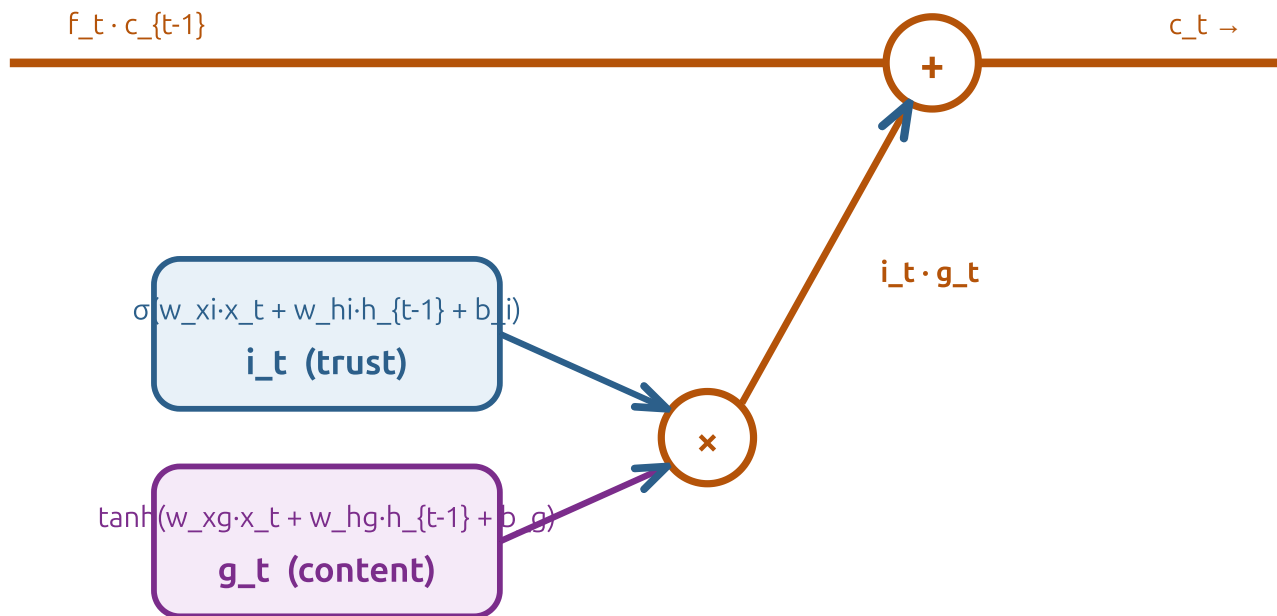
STEP 3 — THE FULL CELL UPDATE: KEEP + ADD

Plain English first: "*new ledger = what we decided to keep, plus what we decided to add.*" Then the equation:

$$c_t = \underbrace{f_t \cdot c_{t-1}}_{\text{what we keep}} + \underbrace{i_t \cdot g_t}_{\text{what we add}}$$

The added amount is a product: *content* g_t times *trust* i_t . A bold claim we don't trust (g_t large, i_t small) barely moves the ledger; a modest claim we fully trust goes in cleanly. (With vectors these products are element-wise — each memory slot has its own valve — but one slot at a time is all we need today.)

INPUT GATE + CANDIDATE → the write



Analogy: a newsroom. The **candidate** is the reporter's draft — it can praise or criticize (positive or negative), but house style caps its intensity (bounded by $\tanh$). The **input gate** is the editor's red pen — a well-sourced story runs at nearly full strength ($i_t \approx 1$); a flimsy rumor gets cut to a footnote ($i_t \approx 0$). What lands in the permanent record is always *draft* $\times$ *editorial trust*.

Which failure this solves: Attempt #2's. The sliding window kept things by *age*; the input gate keeps things by *importance*. "The dip comes in December" can be written into the ledger with high trust and then ride the belt safely for twelve months — while "Tuesday was cloudy" gets gated down to nearly nothing at the moment of writing.

5.3 · The Output Gate — knowing versus saying

The intuition, simply: the ledger is our private knowledge; the prediction is what we say out loud today. Those aren't the same thing. We might be carrying the December-dip fact for months before it becomes relevant — the ledger should hold it quietly, and only *voice* it when the moment comes. One more valve.

STEP 1 — THE VALVE (SAME TRUSTED RECIPE, THIRD SET OF WEIGHTS)

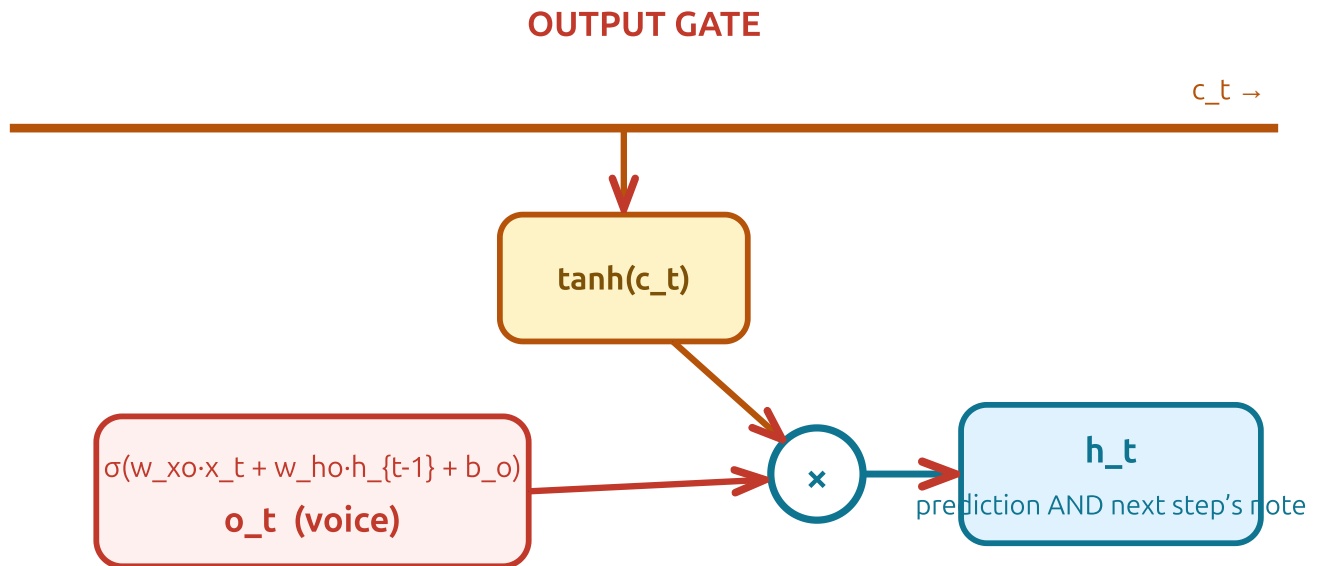
$$\text{net}_o = w_{xo} x_t + w_{ho} h_{t-1} + b_o \quad \Rightarrow \quad o_t = \sigma(\text{net}_o)$$

STEP 2 — THE SPOKEN ANSWER: THE NEW WORKING NOTE

Squash the ledger to a civilized range with $\tanh$, then let the valve decide how much of it escapes:

$$h_t = o_t \cdot \tanh(c_t)$$

And notice the loop close: this h_t is exactly the “yesterday’s working note” that feeds all four weighted sums at the next step. The short-term memory isn’t a separate gadget bolted on — it *is* the gated glimpse of the long-term ledger, refreshed every step.

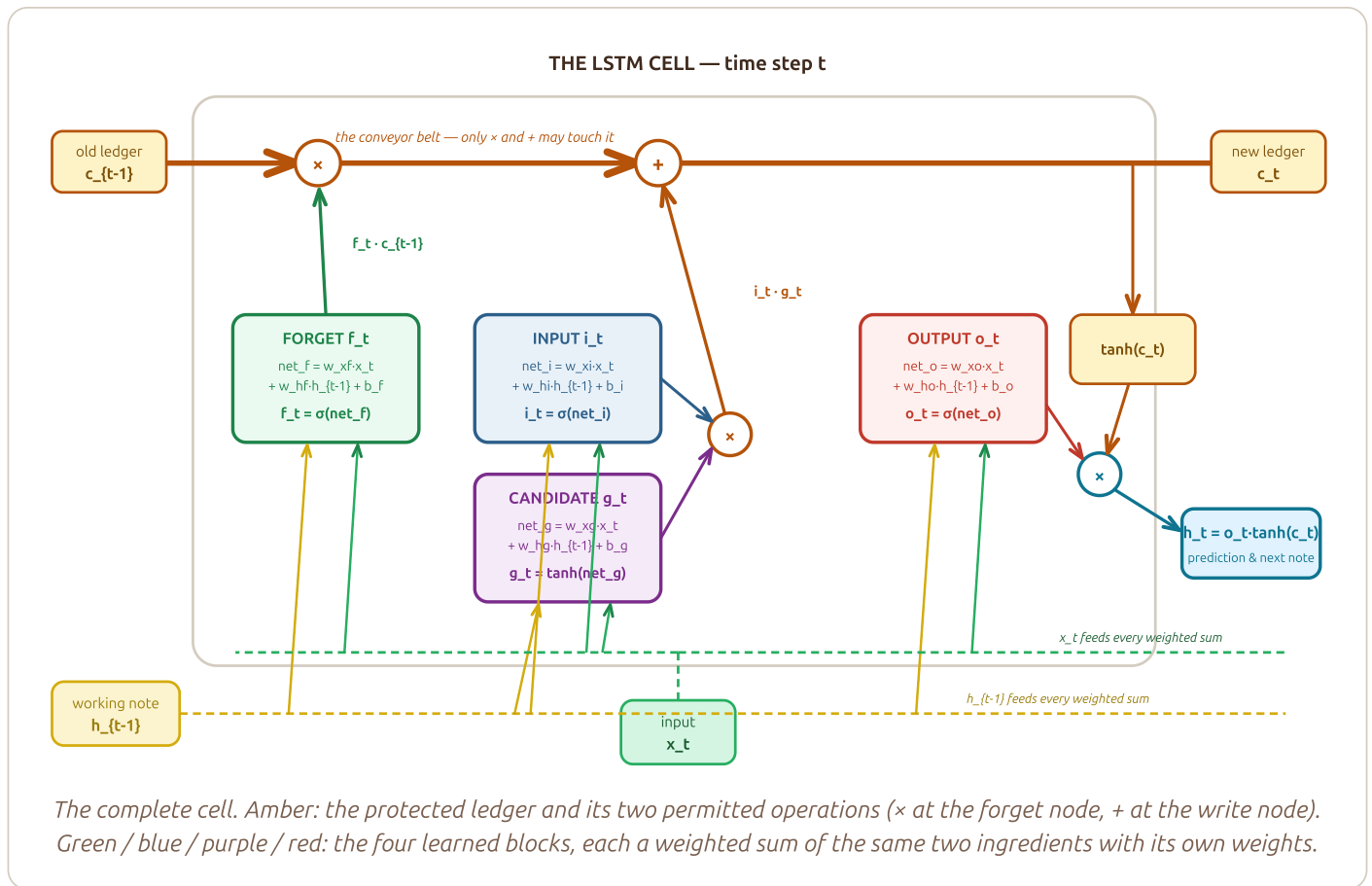


Analogy: a courtroom witness who knows the whole case file (c_t) but answers only the question actually asked. o_t is the discipline between what is *known* and what is *said*. Some days the file is rich but barely relevant — speak softly (o_t small). Other days one page of it is the whole answer — open up ($o_t \rightarrow 1$).

Which failure this solves: the *relevance* half of Attempt #2’s problem. A short-term-only memory was forced to make its stored past and its current answer the same thing. Splitting “stored” (c_t) from “spoken” (h_t) means the machine can carry information for many steps *without* polluting every intermediate prediction with it — storage and speech get separate controls.

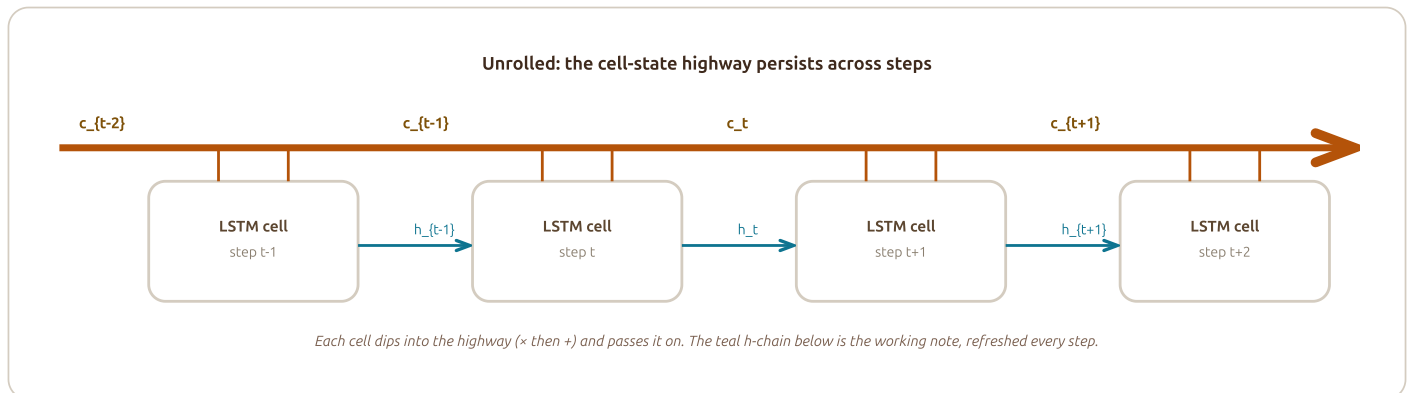
6 • The Complete Cell — Everything We Built, in One Machine

Time to assemble. Every piece below is something we invented at this table for a specific reason: the amber belt on top is the protected ledger, the three sigmoid valves and one tanh proposal sit underneath it, and the bottom rail carries the two ingredients (x_t and h_{t-1}) that feed every weighted sum.



The highway, unrolled across time

Now zoom out. Chain the cell across four time steps and watch the amber line: it runs straight through every step, touched only by a multiply and an add each time. That unbroken line is what lets December's clue survive to next December.



Legend — the naming convention (identical to the backprop document)

Every weight is $w_{\square\Delta}$: the first subscript letter says *what it multiplies* (x = today's input, h = yesterday's note), the second says *which block it belongs to* (f forget, i input, g candidate, o output). Each block also has a bias $b_{\square}$ and a raw score $\text{net}_{\square}$. When we pin a computation to a specific time step, the whole name gets a time subscript: $w_{x o_{t-1}}$ is the output gate's input-weight as used at step $t-1$.

7 • The Numbers — One Real Forward Pass

Talk is cheap; let's push one actual number through the machine. We use the exact values from the companion backprop document so the two references line up digit for digit. The situation: a fresh start ($h_{t-2} = 0$, $c_{t-2} = 0$), one input $x_{t-1} = 0.5$, and a known correct answer, target = 0.8.

The twelve parameters (as used at step t-1)

Forget: $w_{xf_{t-1}} = 0.5$, $w_{hf_{t-1}} = 0.4$, $b_{f_{t-1}} = 0.1$ | Input: $w_{xi_{t-1}} = 0.3$, $w_{hi_{t-1}} = 0.6$, $b_{i_{t-1}} = -0.2$ | Candidate: $w_{xg_{t-1}} = 0.7$, $w_{hg_{t-1}} = 0.2$, $b_{g_{t-1}} = 0.3$ | Output: $w_{xo_{t-1}} = 0.4$, $w_{ho_{t-1}} = 0.5$, $b_{o_{t-1}} = -0.1$

First: what would our two failed predictors say?

Long-term only (cumulative mean)

With one observation, its guess is stuck at the average of what it has seen — and it has *no mechanism* to weight today more than any ancient day. Fine now; hopeless the moment the world shifts.

Short-term only (last value)

Its guess is just $x_{t-1} = 0.5$ — and it will still be “whatever happened most recently” forever, with no way to carry a season-old clue toward the 0.8 target.

The LSTM starts with the same information — but every judgement below is made by *learnable* valves, so training can push each one toward whatever this data needs.

Step 1 — Forget gate

RAW SCORE, THEN SQUASH

$$\text{net}_f = w_{xf_{t-1}} x_{t-1} + w_{hf_{t-1}} h_{t-2} + b_{f_{t-1}} = (0.5)(0.5) + (0.4)(0) + 0.1 = \mathbf{0.35}$$

$$f_{t-1} = \sigma(0.35) = \frac{1}{1 + e^{-0.35}} = \mathbf{0.5866}$$

The valve is 58.66% open. It scales the old ledger: $f_{t-1} \cdot c_{t-2} = 0.5866 \times 0 = 0$ — nothing to keep yet, and the gate handles that gracefully.

Step 2 — Input gate and candidate

TWO BLOCKS, SAME RECIPE, DIFFERENT WEIGHTS

$$\text{net}_i = (0.3)(0.5) + (0.6)(0) + (-0.2) = \mathbf{-0.05} \quad i_{t-1} = \sigma(-0.05) = \mathbf{0.4875}$$

$$\text{net}_g = (0.7)(0.5) + (0.2)(0) + 0.3 = \mathbf{0.65} \quad g_{t-1} = \tanh(0.65) = \mathbf{0.5717}$$

The candidate proposes +0.5717; the editor trusts 48.75% of it. The write is $i_{t-1} \cdot g_{t-1} = 0.4875 \times 0.5717 = \mathbf{0.2787}$.

Step 3 — Update the ledger

KEEP + ADD

$$c_{t-1} = f_{t-1} \cdot c_{t-2} + i_{t-1} \cdot g_{t-1} = 0 + 0.2787 = \mathbf{0.2787}$$

Step 4 — Output gate, and speak

VALVE, TAP, MULTIPLY

$$\text{net}_o = w_{xo_{t-1}} x_{t-1} + w_{ho_{t-1}} h_{t-2} + b_{o_{t-1}} = (0.4)(0.5) + (0.5)(0) + (-0.1) = \mathbf{0.1}$$

$$o_{t-1} = \sigma(0.1) = \mathbf{0.5250} \quad \tanh(c_{t-1}) = \tanh(0.2787) = \mathbf{0.2717}$$

$$h_{t-1} = o_{t-1} \cdot \tanh(c_{t-1}) = 0.5250 \times 0.2717 = \mathbf{0.1426}$$

Step 5 — How wrong were we?

PLAIN ENGLISH, THEN THE LOSS

We want a single number measuring how far the spoken answer (0.1426) landed from the truth (0.8). Take the difference and square it — squaring makes the number positive either way and punishes big misses more than small ones:

$$L = (\text{target} - h_{t-1})^2 = (0.8 - 0.1426)^2 = \mathbf{0.4321}$$

Where the story continues: a loss of 0.4321 is the starting gun for learning. The full gradient calculation for $w_{xo_{t-1}}$ — the exact path with its four derivatives, $(-1.3147)(0.2717)(0.2494)(0.5) = -0.0445$, and the update $0.4 \rightarrow 0.4045$ — is worked step-by-step in the companion backpropagation document, with the same numbers and the same names you just used here.

8 • The History — We Were Not the First at This Table

Here is the satisfying part: the path we just walked — naive memory, opposite failures, protected belt, learned valves — is essentially the path the field walked in real life, just slower and with more pain.

Through the late 1980s and early 1990s, researchers trained recurrent networks shaped like our first recurrence: one memory, re-weighted and squashed at every step. And they kept hitting the same wall. When training tried to send its error signal backward through time, that signal passed through every step's weights and squashing functions — getting multiplied by a factor at each hop. A factor slightly below 1, compounded over 50 steps, is essentially zero: the **vanishing gradient**. Slightly above 1, and it blows up instead: the **exploding gradient**. Either way, the network could not learn that something 50 steps ago mattered — the message from the future never survived the trip back. Sepp Hochreiter had analyzed this precisely in his 1991 thesis; it was the central obstacle of the field.

The people who first solved it — Hochreiter and Jürgen Schmidhuber, in their 1997 paper introducing *Long Short-Term Memory* — realized the fix had to be architectural. Their idea was the very belt we drew in Section 5.0: a cell state whose step-to-step self-connection is fixed at 1, so the error signal riding backward along it is

multiplied by 1 at every hop — neither shrinking nor exploding, circling as long as needed. They called this the **constant error carousel**, and it is the reason our sacred rule (“only $\times$ and $+$ may touch the belt”) exists. The gates came with it: input and output gates in the original design to control the carousel’s traffic, with the forget gate added shortly after (Gers, Schmidhuber & Cummins, 1999) when it became clear the carousel also needed a drain — our whiteboard insight #1, discovered in production.

So every oddity of the architecture is a scar from a specific old wound: the untouched belt exists because gradients used to vanish; the gates exist because an untouched belt is useless without controlled traffic; the forget gate exists because eternal memory turned out to be a bug, not a feature. Nothing in the LSTM is decoration.

9 • Takeaways — The Mental Model to Keep

Forget gate $f_t = \sigma(\text{net}_f)$

Solves: Attempt #1 — the soup pot with no drain. One learned multiply can erase any amount of stale ledger the moment the world changes.

Input gate + candidate $i_t \cdot g_t$

Solves: Attempt #2’s indiscriminate writing. Content (g_t , signed, bounded) times trust (i_t) means the ledger stores by *importance*, not by age.

Output gate $h_t = o_t \cdot \tanh(c_t)$

Solves: the relevance problem. Knowing and saying get separate controls — carry a clue silently for months, voice it only when it matters.

Cell state belt $c_t = f_t c_{t-1} + i_t g_t$

Solves: vanishing gradients. Only $\times$ and $+$ touch the belt, so the error signal can ride it backward across many steps — the constant error carousel.

The one-sentence mental model: an LSTM is a protected ledger on a conveyor belt, with three learned valves — a drain (f_t), a pen with an editor ($i_t \cdot g_t$), and a voice (o_t) — each one a tiny neuron reading today’s input and yesterday’s note.

You built every part of it at this table by fixing one concrete failure at a time. And because this document and the backpropagation companion share identical names — every $w_{x o_{t-1}}$, every net_f , every b_i — you can

now move seamlessly between *why the gates exist* (here) and *how the gradients flow* (there). That pairing is the whole game: intuition first, then the calculus lands on ground that is already familiar.

“How We Invented the LSTM at the Kitchen Table” — companion to the backpropagation walkthrough. Identical weight naming throughout: w_{xf}, w_{hf}, b_f | w_{xi}, w_{hi}, b_i | w_{xg}, w_{hg}, b_g | w_{xo}, w_{ho}, b_o .